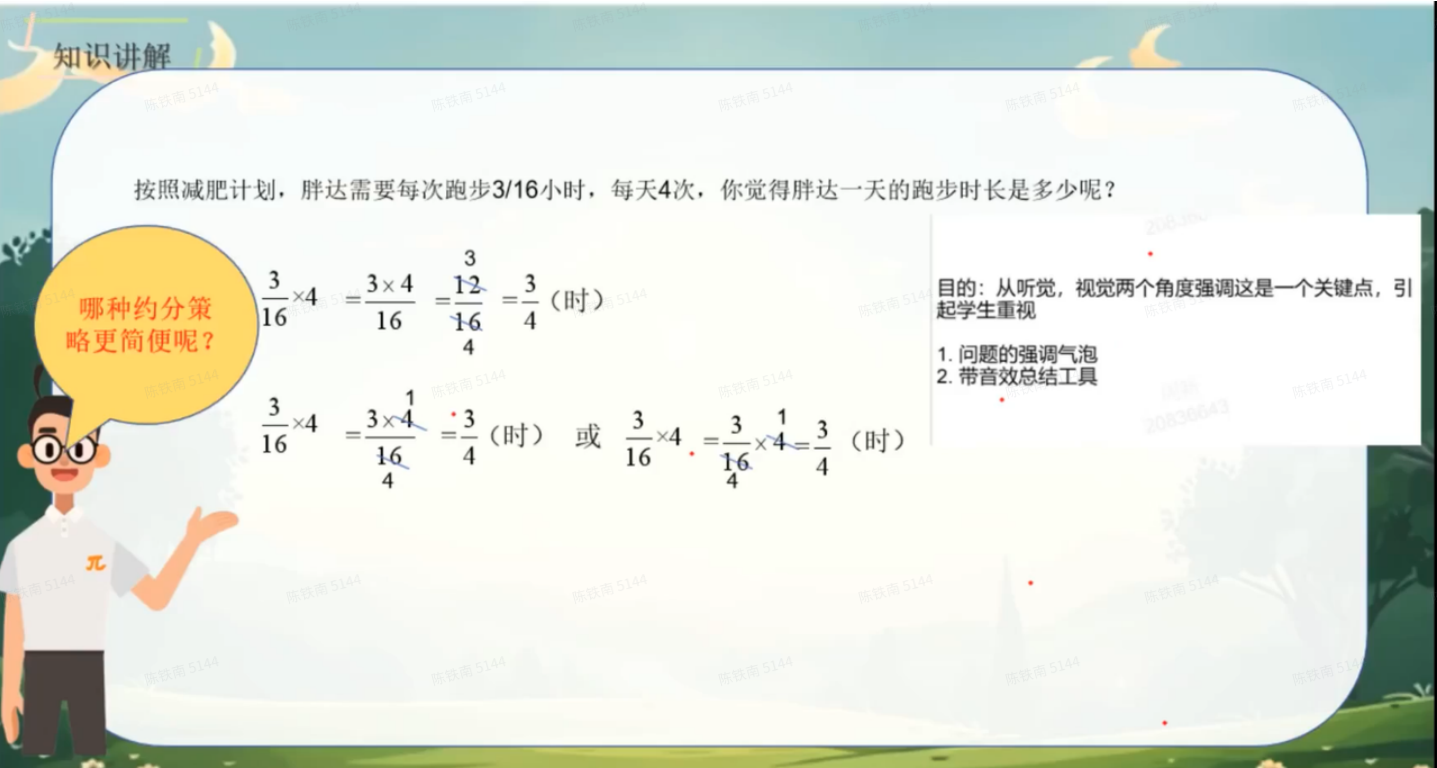


H5课件

问题描述

具体问题

在“知识讲解”(知识讲解环节示例视频)环节中，老师有非常强的流程和动画设计需求。当前的实现更多是说到这里出到哪里到这里的灵活应对，做不到**强设计**。



如上图可见所示，老师期待针对一道例题依次展示”先计算后约分”、“先列分子乘整数再约分”、“先约分后计算”三种计算策略。这样的讲解设计感是极强的，且展示效果也可能是要求非常丰富的，为此引出了本篇文章。

思维提升

数学课上，老师带领大家“回头看”乘法计算的道理。乐乐小组写出了这样两组算式，发现了整数乘法和小数乘法计算道理之间的联系。

$$30 \times 40 = (10 \times 3) \times (10 \times 4) = (10 \times 10) \times (3 \times 4) = 100 \times 12$$

$$0.3 \times 0.4 = (0.1 \times 3) \times (0.1 \times 4) = (0.1 \times 0.1) \times (3 \times 4) = 0.01 \times 12$$

乐乐小组会怎样表达分数乘法的计算道理呢？请以 $\frac{3}{4} \times \frac{2}{3}$ 为例，写一写

$$30 \times 40 = (10 \times 3) \times (10 \times 4) = (10 \times 10) \times (3 \times 4) = 100 \times 12$$

$$\begin{array}{rcl} 30 & = & 10 \times 3 \\ & \downarrow & \downarrow \\ & \text{计数单位} & \text{个数} \\ 40 & = & 10 \times 4 \\ & \downarrow & \downarrow \\ & \text{计数单位} & \text{个数} \end{array}$$

抽象描述

在做课程开发时，碰到了一对冲突：

- 高交互效果和UI灵活应对的矛盾。
- 高流程遵循和响应灵活应对的矛盾。

做答疑的思路一直是灵活应对作为第一优先级。

当前的做法是在不降低响应灵活性的前提下，尽力去满足高交互效果和高流程遵循。从这个过程中，识别到了一种拉扯，尝试用一个工具解决所有问题。

其本质是，一个具体环节的完成，学生参与修改讲解状态的程度，会很大程度的影响效果流程与灵活应对的选择。

! 无法导入该图片，请从原文档中保存原图后重新上传。

举个例子：希望在表达式上做出下图带箭头的演示效果

解决方案-帧动画模式

提供**更多的工具**选择，来解决不同的问题。

目标：提供一个**匹配H5课件动画能力**的定义模式。提供一个更明确的**课件动作定义 + 动作顺序定义 + 动作绑定动画的模式**。让所有组件都可以定义更高的UI效果。

具体做法

对于一个组件，组件中**定义action sequence**，而**action中描述终止状态**，则都可以使用帧动画模式。

模型侧定义组件和动作顺序

代码块

```
1  # 课件区定义
2  <component id="..." desc="组件定义" state="组件状态描述">
3      <action_sequence>
4          <action id="..." cmd="展示分数2/3" target_state="...">
5          <action id="..." cmd="展示分数4/6" target_state="...">
6      </action_sequence>
7  </component>
8
9  # 大纲
10  1. ...
11  ...
12  n. 用id为123的组件，**按照其中action_sequence的顺序，讲解xxxx**。
```

工程侧定义动作和动画的绑定关系

代码块

```
1  bind("component_id.action_sequence.action_id", 具体的动画效果)
```

表达式示例

组件及动作定义参考自善琳的组件定义方式：<https://www.notion.so/kanyun-innovation/action-diff-21e6f7bc7666801aad90cc6494c57f7f>

模型侧

代码块

```
1 <expression id="2-1" >
2   <defs>\def\reduction#1#2{\overset{#1}{\bcancel{#2}}}</defs>
3   <part name="part1" final-state="$12\times\dfrac{3}{4}$" >
4     **<action** name="展示" to="$12\times\dfrac{3}{4}$" />
5   </part>
6   <part name="part2" final-state="$=12\times\dfrac{3}{4}$" >
7     **<action** name="展示" to="$=12\times\dfrac{3}{4}$" />
8     **<action** name="对12约分,对4约分" to="$=\reduction{3}{12}\times\dfrac{3}{\reduction{1}{4}}$" />
9   </part>
10  <part name="part3" final-state="$=9$" >
11    **<action** name="展示" from="" to="$=9$" />
12  </part>
13  <part name="part4" final-state="$ (天) $" >
14    **<action** name="展示" from="" to="$ (天) $" />
15  </part>
16  **<sequential_actions>**
17    <action full_name="2-1.part1.展示">
18    ...
19    <action full_name="2-1.part4.展示">
20  **</sequential_actions>**
21 </expression>
```

工程侧

代码块

```
1 //def bind(action, animation)...
2 bind("2-1.part1.展示", 动画1)
```

Pros

1. 可以支持足够高的UI效果上限
1. 可以提供有一个有稳定预期的动画播放序列，对于制作课件来说会有更好的契约
1. 可以短期内支持任意复杂度的演示效果，且无需介入演示逻辑本身

Cons

1. 在应对灵活性上受限，最好也只能做到尽快从学生的问题中~~抹~~~回到预定义的动作序列上。
~~只能顺序播放动画。

Q&A

1. 是不是所有场景都是这样顺序播放动作的表现呢？
1. 不是的，设计预期是只有明确定义了sequential_actions的环节才会顺序播放。其他未定义sequential_actions的环节不会被影响，还是跟随讲解语义出现。
1. 如果顺序定义的action，前序的action没有生成，怎么处理？
1. 如果前序action没有生成，可以考虑加速播放前序未显示的action。这也是顺序性的价值所在。